

Encoder–IMU Fusion for Stance-Leg Odometry

A Worked Derivation

Arda Gencer

1 What we actually have, and why neither source is enough alone

At every control tick we have two independent raw readings: the three joint encoder angles per leg, $(\theta_{\text{abad}}, \theta_{\text{hip}}, \theta_{\text{knee}})$, and the IMU’s orientation, from which we extract pitch θ and roll φ . Neither one by itself tells us what we actually want, which is the body’s velocity and height in the world frame.

The encoders alone are not enough: forward kinematics on the joint angles tells us exactly where a foot sits relative to the body’s own reference frame, but it says nothing about which way that frame is tilted relative to the ground. A leg held at a fixed configuration produces the exact same joint angles whether the robot is standing level or pitched forward by 30° – the encoders cannot see the difference.

The IMU alone is not enough either: pitch and roll tell us how the body frame is tilted, but on their own they say nothing about where the feet are or whether the robot is translating at all. Orientation and position are simply different quantities.

So the two sources are complementary in a very literal sense – one gives a vector, the other gives the rotation needed to place that vector correctly in the world. Fusing them here means exactly that: computing a vector from the encoders, then rotating it with the IMU’s angles. Nothing probabilistic is happening; this is a deterministic geometric composition, not a Kalman-style statistical fusion.

2 Step 1 – encoders give a body-frame vector

Forward kinematics (the algebraic inverse of the leg’s inverse-kinematics solver) converts one leg’s joint angles into the foot’s position relative to that leg’s ABAD pivot, in the body’s own axes:

$$(f_x, f_y, f_z) = \text{FK}_{3D}(\theta_{\text{abad}}, \theta_{\text{hip}}, \theta_{\text{knee}}; o_y, s, L_1, L_2, D_{\text{ABAD}}) \quad (1)$$

where L_1, L_2 are the leg’s link lengths, D_{ABAD} is the fixed ABAD-to-hip arm length, and o_y, s encode which side (left/right) and end (front/back) of the robot the leg is mounted on. Adding that leg’s fixed mounting offset (x_0, y_0) from the body origin gives the foot position relative to the body origin itself,

$$\mathbf{p}_{\text{foot}}^B = (f_x + x_0, f_y + y_0, f_z). \quad (2)$$

The superscript B is deliberate: this vector is only correct when read in body-frame axes. We do not yet know how those axes sit relative to the world.

3 Step 2 – the IMU supplies the missing rotation

All three orientation angles – pitch θ , roll φ , and yaw ψ – come straight from the IMU’s orientation quaternion. All three are needed: leaving any one out means the rotation only correctly places a

vector for whichever value of that angle happens to be zero, and silently misplaces it otherwise. Section 4 says more about where these three angles come from and what using them here does and does not assume. The body-to-world rotation used is

$$R(\psi, \theta, \varphi) = R_z(\psi) R_y(\theta) R_x(\varphi) = \begin{pmatrix} \cos \psi \cos \theta & \sin \varphi \sin \theta \cos \psi - \sin \psi \cos \varphi & \sin \varphi \sin \psi + \sin \theta \cos \varphi \cos \psi \\ \sin \psi \cos \theta & \sin \varphi \sin \psi \sin \theta + \cos \varphi \cos \psi & \sin \psi \sin \theta \cos \varphi - \sin \varphi \cos \psi \\ -\sin \theta & \sin \varphi \cos \theta & \cos \varphi \cos \theta \end{pmatrix}, \quad (3)$$

with θ = pitch, φ = roll, ψ = yaw (composed as roll, then pitch, then yaw – matrix composition applies right to left, so a body-frame vector is rotated by roll first, then pitch, then yaw). Applying it to the encoder-derived vector re-expresses the same physical offset in world-frame axes:

$$\mathbf{p}_{\text{foot}}^W = R(\psi, \theta, \varphi) \mathbf{p}_{\text{foot}}^B = (w_x, w_y, w_z). \quad (4)$$

This single matrix multiplication is the entire fusion step: an encoder-derived vector, rotated by an IMU-derived matrix.

4 Where the orientation angles come from, and their limits

It is worth being precise about what (ψ, θ, φ) actually are in this implementation, since it affects how the whole estimator should be read. All three are taken directly from the IMU’s orientation quaternion – that is, from the simulator’s exact ground-truth attitude, not from a value derived through any sensor-fusion process. Using the full rotation is still the mathematically correct thing to do with that reading, regardless of where it came from: a rotation matrix missing any one of the three angles only places a vector correctly when that particular angle happens to be zero, and misplaces it – specifically, misdirects horizontal velocity and position, since height is unaffected by yaw entirely (Section 5) – by however much it actually isn’t.

Whether this orientation is realistically obtainable is a separate question worth answering honestly. On real hardware, roll and pitch are reasonably recoverable on their own, since gravity gives a fixed, known reference direction that a stationary or slowly-moving accelerometer can sense. Yaw is a different matter: Bloesch et al. (*State Estimation for Legged Robots – Consistent Fusion of Leg Kinematics and IMU*, RSS 2012) show that yaw and absolute position together form a manifold that is *provably unobservable* from IMU and leg kinematics alone – only roll, pitch, and body velocity can be recovered from that sensor set. Recovering yaw for real would require an additional absolute reference this project does not have, such as a magnetometer (workable in principle, but vulnerable in practice to interference from the leg actuators’ own motor currents) or a vision/LIDAR-based heading estimate. So this estimator should be read precisely for what it is: a correct treatment of a known orientation, where that orientation itself remains a simulation privilege rather than something this sensor suite could produce on a real robot.

5 Height falls out of a flat-ground constraint

If the ground is flat at world $z = 0$ and the current foot is planted on it, then the body’s height plus the foot’s world-frame z -offset from the body must sum to zero:

$$z_{\text{body}} + w_z = 0 \implies z_{\text{body}} = -w_z. \quad (5)$$

Notice this is a single algebraic read-out of Step 2’s result – nothing is integrated, so this estimate carries no accumulating drift; each tick is a fresh, independent computation. It is also, conveniently,

completely insensitive to yaw: a rotation about the vertical axis only ever mixes the x and y components of a vector into each other, it can never move anything into or out of z . So of the three orientation angles discussed in Section 4, only pitch and roll matter here – yaw affects horizontal velocity and position only.

6 Velocity comes from differentiating a constancy constraint

The physical statement that gives us velocity is: while a foot is planted and not slipping, its world position does not move. Writing the body’s world position as $\mathbf{p}_{\text{body}}^W(t)$,

$$\mathbf{p}_{\text{body}}^W(t) + R(t) \mathbf{p}_{\text{foot}}^B(t) = \text{constant}, \quad (6)$$

where $R(t)$ is shorthand for the full rotation of Eq. (3), evaluated at whatever (ψ, θ, φ) the IMU reports at time t . Differentiating both sides with respect to time,

$$\mathbf{v}_{\text{body}}^W(t) = -\frac{d}{dt} \left[R(t) \mathbf{p}_{\text{foot}}^B(t) \right] = - \left[\dot{R}(t) \mathbf{p}_{\text{foot}}^B(t) + R(t) \dot{\mathbf{p}}_{\text{foot}}^B(t) \right], \quad (7)$$

by the product rule. There are genuinely two contributions here: one from the leg’s own joint angles changing ($\dot{\mathbf{p}}_{\text{foot}}^B$), and one from the *body itself rotating* while the foot vector is held ($\dot{R}$). The first term is approximated with a finite difference of the body-frame foot position, exactly as the geometry suggests. The second term is not dropped: using the standard rigid-body identity $\dot{R}(t) = R(t) [\boldsymbol{\omega}^b(t)]_{\times}$, where $\boldsymbol{\omega}^b(t)$ is the body-frame angular velocity the gyro reports directly (the same IMU message already supplies orientation and angular velocity together, so no separate estimation step is needed), this term can be rewritten as

$$\dot{R}(t) \mathbf{p}_{\text{foot}}^B(t) = R(t) \left(\boldsymbol{\omega}^b(t) \times \mathbf{p}_{\text{foot}}^B(t) \right), \quad (8)$$

a cross product computed entirely in body frame, which folds directly into the same finite difference before the single rotation is applied:

$$\mathbf{v}_{\text{body}}^W \approx -R_k \left[\frac{\mathbf{p}_{\text{foot}}^B(t_k) - \mathbf{p}_{\text{foot}}^B(t_{k-1})}{\Delta t} + \boldsymbol{\omega}^b(t_k) \times \mathbf{p}_{\text{foot}}^B(t_k) \right]. \quad (9)$$

This is the only approximation left: $\boldsymbol{\omega}^b$ is evaluated once, at the current tick, rather than integrated exactly across $[t_{k-1}, t_k]$, so a small amount of Δt -discretization error remains – but the rotation-transport term itself is no longer thrown away. A synthetic check confirms this matters in practice: for a body translating at a known velocity while also rotating at a realistic 0.1–0.6 rad/s, dropping this term (as an earlier version of this estimator did) produced a velocity error of about 0.17 m/s on a true speed of 0.37 m/s; including it reduced that error to about 0.0014 m/s, consistent with the residual being pure discretization error rather than a remaining systematic bias.

Once the bracketed term $\mathbf{v}_{\text{rel}}^W = R_k [(\mathbf{p}_k - \mathbf{p}_{k-1})/\Delta t + \boldsymbol{\omega}^b(t_k) \times \mathbf{p}_k]$ is computed, the stance-foot constraint flips its sign: the foot *appearing* to move by $\mathbf{v}_{\text{rel}}^W$ relative to the body means the body itself is moving by exactly the opposite,

$$\mathbf{v}_{\text{body}}^W = -\mathbf{v}_{\text{rel}}^W. \quad (10)$$

7 Worked example

Take the front-left leg ($o_y = +1$, $s = +1$), with $L_1 = L_2 = 0.2$ m, $D_{\text{ABAD}} = 0.1$ m, and mounting offset $(x_0, y_0) = (0.15, 0.113)$ m. Suppose two consecutive ticks, 20 ms apart, read:

	θ_{abad}	θ_{hip}	θ_{knee}		pitch θ	roll φ	yaw ψ
tick $k-1$	0.050	-0.100	0.300	tick $k-1$	0.040	0.015	0.350
tick k	0.050	-0.080	0.290	tick k	0.050	0.020	0.360

The robot has turned about 20° since the estimator started ($\psi \approx 0.35\text{--}0.36$ rad), so this example also shows why the full rotation of Eq. (3) genuinely needs all three angles rather than just pitch and roll.

Step 1. Forward kinematics at each tick gives the foot position relative to the body origin, in body-frame axes – this step never involves the IMU, so it is unchanged from before:

$$\mathbf{p}_{\text{foot}}^B(t_{k-1}) = (0.169767, 0.132618, -0.389523) \text{ m}, \quad (11)$$

$$\mathbf{p}_{\text{foot}}^B(t_k) = (0.175709, 0.132615, -0.389475) \text{ m}. \quad (12)$$

Since the ABAD angle barely changed between ticks (both 0.05 rad) while hip and knee opened up slightly, almost all of the change shows up in f_x – exactly what we’d expect: swinging the hip/knee in their own plane moves the foot mostly along the leg’s forward direction, not sideways.

Step 2. Rotating each by that tick’s own full $R(\psi, \theta, \varphi)$ (Eq. (3)):

$$\mathbf{p}_{\text{foot}}^W(t_{k-1}) = (0.097319, 0.182905, -0.393968) \text{ m} \quad \Rightarrow \quad z_{\text{body}} = 0.393968 \text{ m}, \quad (13)$$

$$\mathbf{p}_{\text{foot}}^W(t_k) = (0.096698, 0.186390, -0.395043) \text{ m} \quad \Rightarrow \quad z_{\text{body}} = 0.395043 \text{ m}. \quad (14)$$

Both height values agree with each other to well within a millimeter despite the body having pitched and rolled slightly between ticks, which is exactly the point of rotating before reading off z rather than reading f_z directly. Consistent with Section 4’s point about yaw and height, recomputing this same example at $\psi = 0$ instead leaves z_{body} completely unchanged at both ticks – only w_x and w_y move, since a 20° heading change substantially redistributes a vector between x and y .

Step 3 – velocity, before the rotation correction. The body-frame finite difference is unchanged from Step 1 (it doesn’t involve the IMU either):

$$\frac{\mathbf{p}_{\text{foot}}^B(t_k) - \mathbf{p}_{\text{foot}}^B(t_{k-1})}{\Delta t} = (0.297093, -0.000119, 0.002378) \text{ m/s}. \quad (15)$$

Step 4 – the rotation-transport correction. This example’s pitch, roll, and yaw all change noticeably from one tick to the next, so it is also a useful check of $\boldsymbol{\omega}^b \times \mathbf{p}_{\text{foot}}^B$. In place of a real gyro sample, approximate $\boldsymbol{\omega}^b$ here by finite-differencing the tick-to-tick angles themselves (a reasonable stand-in at these small angles, though not what the actual implementation does – it reads $\boldsymbol{\omega}^b$ directly from the IMU message):

$$\boldsymbol{\omega}^b \approx \left(\frac{0.020-0.015}{0.02}, \frac{0.050-0.040}{0.02}, \frac{0.360-0.350}{0.02} \right) = (0.25, 0.50, 0.50) \text{ rad/s}. \quad (16)$$

Crossing this with $\mathbf{p}_{\text{foot}}^B(t_k)$ gives a correction of $(-0.261045, 0.185223, -0.054701) \text{ m/s}$ – *larger* than the finite-difference term itself, and enough to change which direction the combined bracket points:

$$\frac{\mathbf{p}_{\text{foot}}^B(t_k) - \mathbf{p}_{\text{foot}}^B(t_{k-1})}{\Delta t} + \boldsymbol{\omega}^b(t_k) \times \mathbf{p}_{\text{foot}}^B(t_k) = (0.036055, 0.185073, -0.052301) \text{ m/s}. \quad (17)$$

Rotating this corrected bracket by the current tick’s full $R(\psi_k, \theta_k, \varphi_k)$ and flipping sign gives the corrected estimate,

$$\mathbf{v}_{\text{body}}^W = (0.034123, -0.185984, 0.050330) \text{ m/s}, \quad (18)$$

compared with $(-0.277871, -0.104413, 0.012476) \text{ m/s}$ from ignoring the rotation term entirely – not just a small correction here, but one that changes the sign of two of the three components. The reason this particular example is so sensitive is that its angles were originally chosen to change quickly from tick to tick purely to make yaw’s effect on the rotation matrix visible (Step 2); a real gait tick with a slower, steadier body rotation would generally see a smaller correction than this, though how small is an empirical question for the actual logged ω^b , not something to assume.

8 Combining legs, and the final integration

When more than one foot is in stance simultaneously, each produces its own independent $(z_{\text{body}}, \mathbf{v}_{\text{body}}^W)$ estimate via the steps above. These are combined with a plain, unweighted arithmetic mean. It is worth being honest that this is a simplification: a leg near full extension is geometrically more sensitive to a given joint-angle error than one near mid-range, so a statistically rigorous fusion would weight each leg’s contribution by its estimated reliability (e.g. a weighted least-squares or Kalman update) rather than trusting every stance leg equally.

Finally, horizontal position is obtained with a single Euler integration of the fused velocity,

$$x_{\text{body}}(t_k) = x_{\text{body}}(t_{k-1}) + v_x \Delta t, \quad (19)$$

and likewise for y . This is the only integration anywhere in the pipeline – height never accumulates error since it is read fresh each tick, and velocity itself is obtained by differentiation rather than integration, so horizontal position is one integration away from a non-drifting quantity rather than two integrations away from a double-drifting one, as a raw accelerometer-based estimate would be.